Holly Pasquinelli
Lab 9

Static Malware Analysis

Non-technical Summary

In this lab, we explored different ways to analyze malware. When using malware samples, it is important to work on a Linux, or virtual machine to ensure the malware is not run on our physical device. We also explored the difference between static and dynamic malware analysis. Static malware analysis provides methods to learn about different pieces of the malware, without executing it. Dynamic malware analysis is running the malware in a safe and contained environment. We know there are many types of malware, such as viruses, trojan horses, worms, rootkits, and spyware. In digital forensics it is important to understand each type of malware, and their potential targets so it is easier to recognize them when it comes to malware analysis. Using tools such as pefile, readpe, and pe-tree are extremely useful when it comes to learning more about malware, and its parts. Most importantly, we are able to extract information from malware safely without having to execute the program, and put a device at risk.

Technical Report

We started by installing the pefile and readpe libraries. I had to use python3-xyz in order to successfully install the pefile library. We also downloaded the pe-tree tool by cloning the git repository.

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install python3-pefile
[sudo] password for kali:
python3-pefile is already the newest version (2024.8.26-2.1).
python3-pefile set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1228

┌──(kali㉿kali)-[~]
└─$ sudo apt install readpe
readpe is already the newest version (0.85.1-1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1228

┌──(kali㉿kali)-[~]
└─$ git clone http://github.com/blackberry/pe_tree.git
Completing main porcelain command
add                 -- add file contents to index
am                  -- apply patches from a mailbox
archive             -- create archive of files from named tree
```

Once we successfully installed the libraries and tools, we were able to use python3 in Linux. Using python3 gives us a python interpreter, so we are able to run python commands in the Linux command line. Below, we begin to execute commands using the pefile library. Uploading the file “malware2.exe”, we are now able to view a lot of information about “malware2.exe”. This produced an extremely long output. This is essentially a summary of everything in the PE file.

```
>>> import pefile
>>> pe = pefile.PE("malware2.exe")
>>> print(pe.print_info())
---------DOS_HEADER---------

[IMAGE_DOS_HEADER]
0x0         0x0   e_magic:                       0x5A4D
0x2         0x2   e_cblp:                        0x90
0x4         0x4   e_cp:                          0x3
0x6         0x6   e_crlc:                        0x0
0x8         0x8   e_cparhdr:                     0x4
0xA         0xA   e_minalloc:                    0x0
0xC         0xC   e_maxalloc:                    0xFFFF
0xE         0xE   e_ss:                          0x0
0x10        0x10  e_sp:                          0xB8
0x12        0x12  e_csum:                        0x0
0x14        0x14  e_ip:                          0x0
0x16        0x16  e_cs:                          0x0
0x18        0x18  e_lfarlc:                      0x40
0x1A        0x1A  e_ovno:                        0x0
0x1C        0x1C  e_res:
0x24        0x24  e_oemid:                       0x0
0x26        0x26  e_oeminfo:                     0x0
0x28        0x28  e_res2:
0x3C        0x3C  e_lfanew:                      0x120

---------NT_HEADERS---------

[IMAGE_NT_HEADERS]
0x120       0x0   Signature:                     0x4550
```

On the left, we print the DOS Header. This contains important information about the PE file. Using this information, we can verify the file signature and where the file begins. We can see the output of offset from the beginning of the file, offset from the beginning of the header, field name, and value from the first to fourth column, respectively. For example, we can verify that this file is an .exe file because the

```
>>> print(pe.DOS_HEADER)
[IMAGE_DOS_HEADER]
0x0         0x0   e_magic:                       0x5A4D
0x2         0x2   e_cblp:                        0x90
0x4         0x4   e_cp:                          0x3
0x6         0x6   e_crlc:                        0x0
0x8         0x8   e_cparhdr:                     0x4
0xA         0xA   e_minalloc:                    0x0
0xC         0xC   e_maxalloc:                    0xFFFF
0xE         0xE   e_ss:                          0x0
0x10        0x10  e_sp:                          0xB8
0x12        0x12  e_csum:                        0x0
0x14        0x14  e_ip:                          0x0
0x16        0x16  e_cs:                          0x0
0x18        0x18  e_lfarlc:                      0x40
0x1A        0x1A  e_ovno:                        0x0
0x1C        0x1C  e_res:
0x24        0x24  e_oemid:                       0x0
0x26        0x26  e_oeminfo:                     0x0
```

```
>>> print(pe.DOS_HEADER.dump_dict())
{'Structure': 'IMAGE_DOS_HEADER', 'e_magic': {'FileOffset': 0, 'Offset': 0, '
Value': 23117}, 'e_cblp': {'FileOffset': 2, 'Offset': 2, 'Value': 144}, 'e_cp
': {'FileOffset': 4, 'Offset': 4, 'Value': 3}, 'e_crlc': {'FileOffset': 6, 'O
ffset': 6, 'Value': 0}, 'e_cparhdr': {'FileOffset': 8, 'Offset': 8, 'Value':
4}, 'e_minalloc': {'FileOffset': 10, 'Offset': 10, 'Value': 0}, 'e_maxalloc':
 {'FileOffset': 12, 'Offset': 12, 'Value': 65535}, 'e_ss': {'FileOffset': 14,
 'Offset': 14, 'Value': 0}, 'e_sp': {'FileOffset': 16, 'Offset': 16, 'Value':
 184}, 'e_csum': {'FileOffset': 18, 'Offset': 18, 'Value': 0}, 'e_ip': {'File
Offset': 20, 'Offset': 20, 'Value': 0}, 'e_cs': {'FileOffset': 22, 'Offset':
22, 'Value': 0}, 'e_lfarlc': {'FileOffset': 24, 'Offset': 24, 'Value': 64}, '
e_ovno': {'FileOffset': 26, 'Offset': 26, 'Value': 0}, 'e_res': {'FileOffset'
: 28, 'Offset': 28, 'Value': '\\x00\\x00\\x00\\x00\\x00\\x00\\x00\\x00'}, 'e_
oemid': {'FileOffset': 36, 'Offset': 36, 'Value': 0}, 'e_oeminfo': {'FileOffs
et': 38, 'Offset': 38, 'Value': 0}, 'e_res2': {'FileOffset': 40, 'Offset': 40
, 'Value': '\\x00\\x00\\x00\\x00\\x00\\x00\\x00\\x00\\x00\\x00\\x00\\x00\\x00
\\x00\\x00\\x00\\x00\\x00\\x00\\x00'}, 'e_lfanew': {'FileOffset': 60, 'Offset
': 60, 'Value': 288}}
```

signature matches 5A4D. If we ran a .exe, and we did not see this signature, the file may not actually be an executable, or it may be corrupted.

On the right, we convert the DOS header information to a format that Python is able to read. We are now able to print different fields, and manipulate the data.

```
>>> print(pe.DOS_HEADER.sizeof())
64
>>> print('offset to the PE header:', pe.DOS_HEADER.e_lfanew)
offset to the PE header: 288
>>> print('e_magic in decimal:', pe.DOS_HEADER.e_magic)
e_magic in decimal: 23117
>>> print('e_magic in hexadecimal:', hex(pe.DOS_HEADER.e_magic))
e_magic in hexadecimal: 0x5a4d
>>> print("[*] Signature: " + hex(pe.NT_HEADERS.Signature))
[*] Signature: 0x4550
```

Above, we printed the size of the DOS header, the offset to the PE header, e_magic in hex, and decimal. We also are able to format the output, and make it more human readable.

Below, we printed the file header, which gives us information about the machine CPU such as number of sections, how many bits the system is, and the last time the system was compiled. This system is 0x14C, has 7 sections, and was last compiled Friday March 10, 2023 around 2:25.

```
>>> print(pe.FILE_HEADER)
[IMAGE_FILE_HEADER]
0x124      0x0   Machine:                      0x14C
0x126      0x2   NumberOfSections:             0x7
0x128      0x4   TimeDateStamp:                0x640A9521 [Fri Mar 10 02:25:
37 2023 UTC]
0x12C      0x8   PointerToSymbolTable:         0x0
0x130      0xC   NumberOfSymbols:              0x0
0x134      0x10  SizeOfOptionalHeader:         0xE0
```

Below, we print the information from the data directory field from the optional header in a more readable format. This information is important because it tells us the locations of critical data, as well as tables used in the executable. We can see that we are able to print each value from the data directory, its name, size, and address. We can also see that in this case, the virtual address is printed as a hex value.

```
>>> for val in pe.NT_HEADERS.OPTIONAL_HEADER.DATA_DIRECTORY:
...     print("Name: " + val.name + " | Size: " + str(val.Size) + " | Addres\
s : " + hex(val.VirtualAddress))
...
Name: IMAGE_DIRECTORY_ENTRY_EXPORT | Size: 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_IMPORT | Size: 160 | Address : 0x15758c
Name: IMAGE_DIRECTORY_ENTRY_RESOURCE | Size: 54288 | Address : 0x162000
Name: IMAGE_DIRECTORY_ENTRY_EXCEPTION | Size: 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_SECURITY | Size: 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_BASERELOC | Size: 41032 | Address : 0x170000
Name: IMAGE_DIRECTORY_ENTRY_DEBUG | Size: 56 | Address : 0x14f360
Name: IMAGE_DIRECTORY_ENTRY_COPYRIGHT | Size: 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_GLOBALPTR | Size: 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_TLS | Size: 24 | Address : 0x14f3c0
```

Next, we start looking into the sections of the PE. This for loop prints part of the information for each section. We can see virtual address, virtual size, and size of raw data for each section all in hex values. If we want to see all of the information of a section, we can use the command print(pe.sections[0]), which would print the first section since we are using index 0. This can help us find malicious code or data hidden in the executable based on where the data is located, and its size.

```
>>> for section in pe.sections:
...     print(section.Name.decode().rstrip('\x00'))
...     print(" Virtual Address: " + hex(section.VirtualAddress))
...     print(" Virtual Size " + hex(section.Misc_VirtualSize))
...     print(" Size of raw data " + hex(section.SizeOfRawData) + '\n')
...
.text
 Virtual Address: 0×1000
 Virtual Size 0×124a11
 Size of raw data 0×124c00

.guclvne
 Virtual Address: 0×126000
 Virtual Size 0×508a
 Size of raw data 0×5200

.rdata
 Virtual Address: 0×12c000
 Virtual Size 0×2c74a
 Size of raw data 0×2c800

.data
 Virtual Address: 0×159000
 Virtual Size 0×77e4
 Size of raw data 0×2e00

.guclvne
 Virtual Address: 0×161000
 Virtual Size 0×50
 Size of raw data 0×200
```

Below, we can use this for loop to view all the DLL files the executable has imported, as well as functions it has used. This can help us recognize malware because malware usually imports more libraries and uses more functions than usual.

```
>>> for entry in pe.DIRECTORY_ENTRY_IMPORT:
...     print(entry.dll.decode('utf-8'))
...     for fnc in entry.imports:
...         print("| ", fnc.name.decode('utf-8'))
...
KERNEL32.dll
|  GetComputerNameW
|  GetModuleFileNameA
|  GetCurrentProcessId
|  OpenProcess
|  GetModuleFileNameW
|  SetLastError
|  WaitForSingleObject
|  CreateEventW
|  FreeLibrary
|  WinExec
|  GetPrivateProfileStringW
|  CopyFileW
```

Before using the readpe command, we used some basic Linux commands to verify basic information about the “malware2.exe” file. For example, we can use the file command to display the type of file, system, and number of sections.

The readpe command is a very useful tool when it comes to static malware analysis. Using readpe, we can display full output, all PE headers, all section headers, data directories, specified headers or directories, imported and exported functions, and versions. Using the jq command, we are able to handle JSON objects. So once we display the DOS header in JSON format, we are able to display the magic number using fields. We can see it printed with the correct signature for exe, 5A4D.

```
┌──(env)─(kali㉿kali)-[~/pe_tree]
└─$ readpe -h dos -f json malware2.exe | jq '."DOS Header"."Magic number"'
"0×5a4d (MZ)"
```

Using readpe -i malware2.exe, we can display all the imported functions of the executable. Using the -S flag will display all sections. Below, we used the -d flag to display the data directories. This displays all directories that do not have a size 0. The different flags in the readpe library allow us to examine specific parts of an executable. Once again, this prevents us from being forced to run the executable, and we can examine the file piece by piece safely.

```
┌──(env)─(kali㉿kali)-[~/pe_tree]
└─$ readpe -d malware2.exe
Data directories
    Directory
        IMAGE_DIRECTORY_ENTRY_IMPORT:    0×15758c (160 bytes)
    Directory
        IMAGE_DIRECTORY_ENTRY_RESOURCE:  0×162000 (54288 bytes)
    Directory
        IMAGE_DIRECTORY_ENTRY_BASERELOC: 0×170000 (41032 bytes)
    Directory
        IMAGE_DIRECTORY_ENTRY_DEBUG:     0×14f360 (56 bytes)
    Directory
        IMAGE_DIRECTORY_ENTRY_TLS:       0×14f3c0 (24 bytes)
    Directory
        IMAGE_DIRECTORY_ENTRY_LOAD_CONFIG: 0×14f2a0 (64 bytes)
    Directory
        IMAGE_DIRECTORY_ENTRY_IAT:       0×12c000 (780 bytes)
```

References

1. Lab 15 pdf